Inside the Spec

Real Scenarios. Working Artifacts.
Decisions That Matter.

API не ответил вовремя. Платёж прошёл?

Одно пополнение. Неизвестный результат.
Спецификация дальнейших действий.

CASE 01 · PAYMENT TIMEOUT

€50. Достоверного ответа нет.

Пользователь пополняет баланс. Провайдер мог принять платёж до истечения времени ожидания запроса.

01 **Запрос завершился timeout**
Мы не знаем, создан ли платёж.

02 **Пользователь повторяет действие**
Сохраняем прежнее намерение и клиентский ключ.

03 **Подтверждение приходит позже**
Проверяем результат до изменения платежа.

> **Timeout не доказывает, что платёж завершился отказом.**

Условия кейса: один провайдер; окно идемпотентности 24 часа, включая конкурентные вызовы; поиск по постоянной ссылке; неизменные финальные результаты. Для реальной интеграции условия проверяются отдельно.

Три независимых статуса.

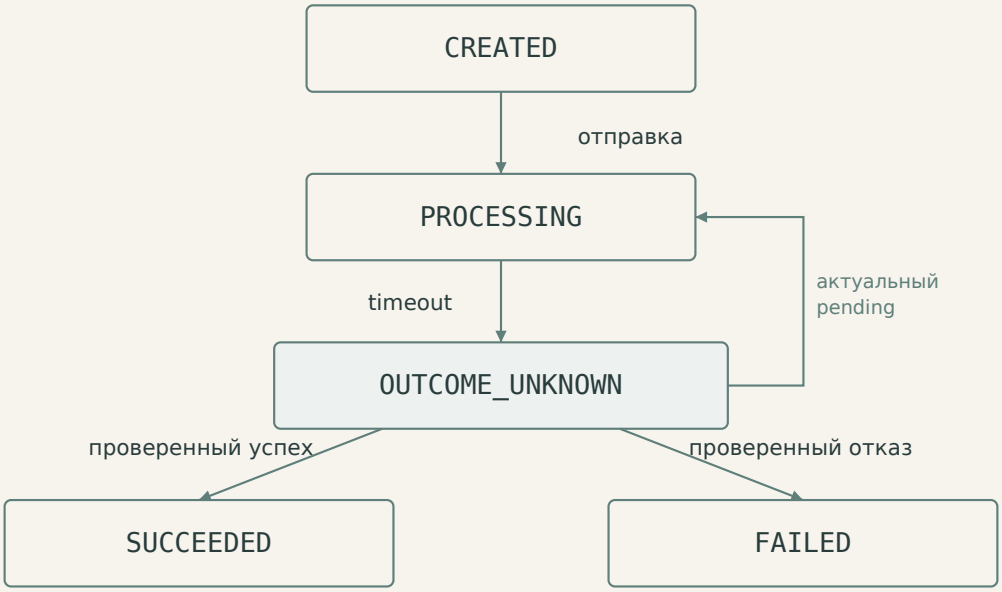
payment_status	SUCCEEDED	Провайдер подтвердил успех платежа.
credit_status	PENDING	Проводка на балансе ещё не подтверждена.
review_status	OPEN	Хотя бы одна причина разбора остаётся открытой.

Эти значения могут существовать одновременно. Общий «статус платежа» скрыл бы, какая часть процесса ещё требует действий.

Ошибка зачисления не делает платёж FAILED. Метка разбора не заменяет известный результат платежа.

Не скрывайте неизвестность.

На схеме показано восстановление после неоднозначного create. Состояния отражают то, что известно нашему сервису.

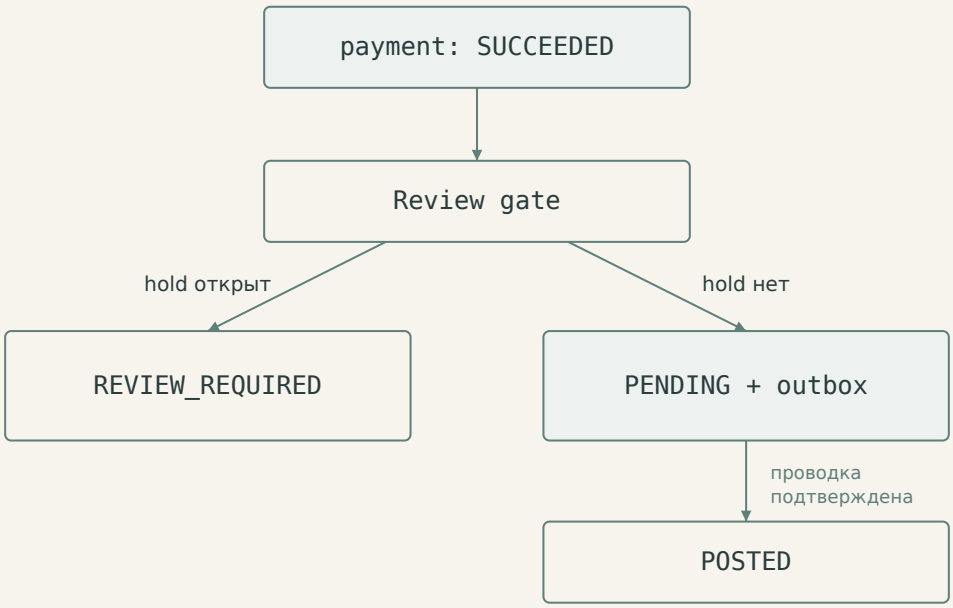


Также: PROCESSING → проверенный SUCCEEDED / FAILED; CREATED → FAILED при окончательном локальном отказе до отправки. Полная таблица в спецификации.

> **Ни NOT_FOUND, ни окончание окна не означают FAILED.**

Свяжите успех и зачисление.

Успех платежа, проверка разбора и задание на зачисление связаны явно.



Платёжная БД локально сохраняет успех и outbox-задание. Проводка позже создаётся транзакцией БД баланса. CR-01/CR-02 задают проверку и границу отправки.

Одно намерение. Те же ID.

Идентификатор	Что он обозначает
client_idempotency_key	Намерение пользователя. Сохраняется при перезагрузке и сбое сети.
payment_id	Одна внутренняя платёжная операция.
provider_idempotency_key	Та же операция провайдера при повторах create.
attempt_id	Один исходящий вызов CREATE или LOOKUP.

> **Тот же клиентский ключ + те же параметры → существующая операция.**

Область ключа: пользователь + тип операции + ключ. Fingerprint: нормализованные сумма, валюта, инструмент.

Ключ, fingerprint и операция сохраняются атомарно. Уникальность обеспечивается ограничением хранилища.

Опишите повторный запрос.

Условие	Поведение API
Те же ключ и данные; любое нефинальное состояние	202 · Полный PaymentStatus. Далее GET.
Тот же ключ, другие данные	409 · IDEMPOTENCY_KEY_REUSED.
Тот же ключ; платёж финальный	200 · Прежний результат; статус зачисления отдельно.
Новый ключ; совпал незавершённый платёж	409 · Исходный PaymentStatus + SHOW_EXISTING_PAYMENT.
Нельзя надёжно сохранить операцию	503 · Внешний вызов требует подтверждённой записи.

> **Повтор клиента сам по себе не запускает новый create провайдеру.**

Если ответ commit потерян, повтор с прежним ключом находит сохранённую операцию. Смена ключа не является способом восстановления.

202 возвращает объект статуса.

POST использует 202 для всех трёх нефинальных состояний. Ниже фрагмент DTO; полный контракт также содержит credit_status и review_status.

```
payment_id: pay-001
payment_status: PROCESSING
status_url: /payments/pay-001
next_action: WAIT
poll_after_seconds: 5
retry_post: false
```

> **После 202 прекратить POST. Читать ту же операцию через GET.**

GET возвращает 200 и для unknown. Опрос не чаще раза в 5 секунд, максимум 60 секунд, затем остановка и действие обновления.

Открытый разбор даёт CONTACT_SUPPORT. Guard нового ключа возвращает полную исходную операцию и SHOW_EXISTING_PAYMENT. HTTP 200 сам по себе не означает успех платежа.

Ограничьте восстановление.

```
Timeout CREATE / LOOKUP: 10 секунд
LOOKUP: +15с, +1м, +5м, +15м, +1ч
Бюджет CREATE: первый + 1 повтор
```

При неизвестном результате сначала lookup. NOT_FOUND разрешает повтор только с прежними provider key и параметрами, свободным бюджетом и безопасным окном.

> **Через 1 час нет результата → ручной разбор. Автоматический create останавливается; поздние подтверждения принимаются.**

Окно провайдера: 24 часа. Локальная граница: до 23 часов от первой отправки. Более ранний часовой предел также действует.

Параметры кейса. Без подтверждённой семантики безопасного окна create не повторяется. После часа: ответственный разбор и ежедневная сверка, без нового платежа.

HTTP-код не доказывает результат.

Ответ провайдера	Решение о восстановлении
429; непринятие доказано	Учесть Retry-After. Повтор допустим, если все прежние попытки также доказанно не приняты.
429; такой гарантии нет	Результат неизвестен. До возможного повтора нужен lookup.
502 / 503 / 504 или timeout	Unknown, если контракт не даёт отдельной гарантии.
Документированный финальный отказ	FAILED только при доказанном окончательном отказе.

> **Все вызовы create расходуют общий лимит, включая ответы 429.**

Непринятие второй попытки не разрешает неизвестность первой. Ошибка lookup не определяет результат платежа.

Закрывайте unknown по доказательству.

Подтверждение	Решение
Проверенные финальные успех / отказ	Применить финальный переход; credit gate сохраняется.
Нет строки / NOT_FOUND	Сохранить unknown, guard и владельца.
Финальный NOT_EXECUTED по PA-08	FAILED, если покрыты все попытки и запрещено будущее исполнение.

> **Ежедневная сверка: 02:00 UTC. Эскалация через 24 часа; инцидент через 7 дней. Возраст не создаёт финансовый результат.**

PA-08 является дополнительным условием кейса, а не предполагаемой функцией реального провайдера. Без доказанного неисполнения нельзя ставить FAILED ради очистки очереди.

Новый ключ тоже требует проверки.

Новый ключ блокируется, пока совпадающий платёж не разрешён, независимо от возраста.

> **Те же пользователь, инструмент, сумма и валюта; CREATED, PROCESSING или OUTCOME_UNKNOWN → 409.**

Вернуть полный статус исходной операции и SHOW_EXISTING_PAYMENT. Сначала проверить прежний ключ; сериализовать guard, создание и завершение.

Через 6 минут	Через 24 часа
Unknown сохраняется → новый запрос блокируется.	Unknown сохраняется → новый запрос блокируется.

Истечения по таймеру и обхода пользователем нет. Другие инструменты/суммы и новые намерения после успеха требуют отдельной политики. Ожидание доказательства ограничивает доступность.

У разбора несколько причин.

review_status объединяет отдельные причины разбора. Закрытие одной не должно удалять остальные.

Пришёл поздний проверенный успех	Ожидаемое поведение
Открыт только RESULT_OVERDUE	SUCCEEDED; закрыть просрочку; запланировать зачисление.
Просрочка + SECURITY_HOLD	SUCCEEDED; закрыть только просрочку; удержать зачисление.
Уже сохранён финальный отказ	Сохранить финал; открыть разбор противоречия.

> **Оператор не назначает успех без подтверждения провайдера.**

Агрегат: NONE → OPEN → RESOLVED; новое основание может открыть его снова. У каждой причины есть владелец, следующее действие, доказательства и журнал.

Проверить. Сохранить. Ответить.

Webhook является уведомлением провайдера. Контракт кейса требует проверить исходное тело до применения бизнес-данных.

```
HMAC-SHA256(timestamp + "." + raw_body)
Timestamp доставки: в пределах ±300 с
Дедупликация: provider + event_id
```

Секреты хранятся в secret manager. Проверяются merchant, reference, ID провайдера, сумма и валюта. Подписи сравниваются за постоянное время.

> **2xx только после надёжной записи в inbox. Хранилище недоступно → 503.**

event_id обязателен в подписанном теле. Нет ID → 400, без изменения операции. Повтор упирается в уникальность inbox и в пределах 300 секунд. Новый timestamp доставки не создаёт новое событие.

После успеха нужна проводка.

01 **Сохранить успех платежа**

В той же транзакции создать задание, если нет блокирующего разбора.

02 **Проверить перед отправкой**

Hold удерживает неначатое задание. Снятие последнего hold разрешает доставку.

03 **Обеспечить безопасный повтор**

Баланс атомарно гарантирует одну проводку по payment_id.

> **Timeout или 5xx баланса → повтор того же задания, без нового платежа.**

Нужны backoff с jitter, общие лимиты и circuit breaker. Задания сохраняются; после 24 часов передаются на ручное восстановление.

Hold не отзывает уже начатый вызов. Цель кейса: 99,9% подходящих зачислений за 60 секунд; сигнал через 2 минуты. Знаменатель и исключения заданы в полной спецификации. Измерений пока нет.

Чтение очереди ещё не разрешение.

Перед отправкой: блокировка строки платежа, перечитывание hold, commit DISPATCH_AUTHORIZED. Изменение hold использует ту же строку.

Порядок commit	Последствие
Hold раньше разрешения	Баланс не вызывается, даже если задание уже прочитано.
Разрешение раньше hold	Эта попытка может исполниться. Hold запрещает следующие.
Первый ответ потерян, hold нет	Повторить прежний payment_id; вернуть или создать одну проводку.

> **Атомарность outbox локальная. БД баланса фиксирует проводку отдельно и обеспечивает её уникальность.**

Логическая граница начала: commit разрешения. Последующий hold не гарантирует отзыв вызова. Для строгого отзыва нужен протокол баланса, а не ещё одно локальное чтение.

Второй обработчик перечитывает.

Одновременные подтверждения	Требуемый результат
Lookup PENDING + webhook успеха	SUCCEEDED; старый pending не понижает финал.
Успех из обоих каналов	Один переход, одно логическое задание, одна проводка.
Финальные успех и отказ	Первый допустимый финал сохраняется; разбор конфликта.
Timeout + проверенный успех	Успех сохраняется при любом порядке commit.

Сохранить подтверждение. Проверить версию. Атомарно записать переход и outbox. При конфликте перечитать операцию и заново оценить то же подтверждение.

Локальная блокировка не отменяет начатый сетевой вызов. Пересечение вызовов также требует конкурентной идемпотентности провайдера.

Проверьте неудобные сценарии.

Сценарий	Критерий приёмки
AC-03 · Оба create дали timeout	Unknown сохраняется; третьего create нет.
AC-09 · Поздний успех при security hold	Успех сохранён; закрыта только просрочка; зачисление удержано.
AC-18 · Проводка есть, ответ потерян	Повтор возвращает ту же проводку.
AC-22 · Разные ключи одновременно	Одна операция; второй запрос получает 409.
AC-23 · Новый ключ через 6 минут	409 с исходной операцией; второго платежа нет.

> **В полной спецификации 40 проверочных сценариев со ссылками на правила.**

Это ожидаемые результаты, а не выполненные тесты реализации. Условия провайдера, конкурентность и SLO ещё нужно проверить на реальной системе.

Beyond the Scope

Процесс	Что не описано в этом кейсе
3DS; authorization / capture	Действия пользователя, авторизация и capture.
Refund; reversal; chargeback	Отдельные финансовые операции после успеха платежа.
Смена провайдера; части; FX	Дубликаты между провайдерами, остатки и конвертация.
Адаптеры отчётов провайдера	Форматы и доказательства окончательности.

Отдельно: исправление финалов, строгий отзыв в балансе и дубликаты между инструментами или после финального успеха.

Полная спецификация – см. GitHub.

GitHub Repository for the Inside the Spec

Yulia Melekhova on LinkedIn